

Estrutura de Dados

Tabelas de Dispersão

(hash tables)

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" << endl;
    return 0;
}
```

Tabelas de Dispersão – O que é?

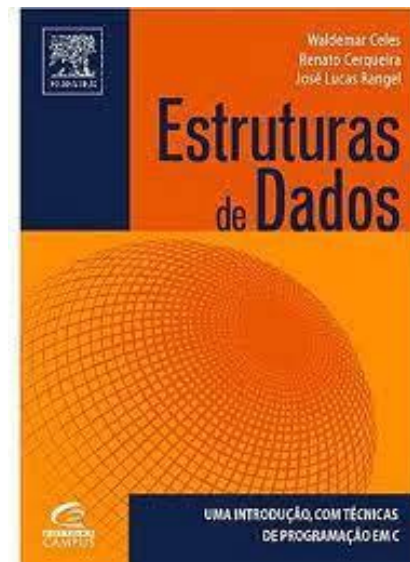
Basicamente pode ser definida como uma estrutura de dados que associa chaves de pesquisa a valores.

O objetivo de uma tabela de Dispersão é possibilitar a busca de dados a partir de uma chave simples formada através de uma **função chamada de *espalhamento***.

Tabelas de Dispersão – Motivação

Segundo W. Celes e J. L. Rangel,

*“ ...tabelas de dispersão são usadas para obtermos algoritmos eficientes, armazenamos os elementos ordenados e tiramos proveito dessa ordenação para alcançar eficientemente o elemento procurado. Chegamos a conclusão que os algoritmos eficientes de busca demandam um esforço computacional de $O(\log n)$. **As estruturas de dados conhecidas como tabelas de dispersão (hash tables), que, se bem projetadas, podem ser usadas para buscar um elemento da tabela em ordem constante: $O(1)$. O preço pago por essa eficiência será um uso maior de memória e é proporcional ao número de elementos armazenados...**”*



Tabelas de Dispersão – Motivação

“ ... Considere que um aluno é individualmente identificado pelo seu número de matrícula. **Podemos então usar o número de matrícula como chave de busca do conjunto de alunos armazenados.**

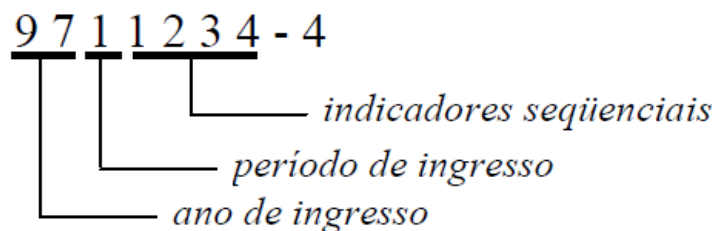
Nesse Exemplo, o número de matrícula dos alunos é dado por uma sequencia de oito dígitos, **Utilizamos nesse caso é o número: 9711234.**

Para permitir um acesso a qualquer aluno em ordem constante, podemos usar o número de matrícula do aluno **como índice de um vetor – vet**. Se isso for possível, acessamos os dados do aluno cuja matrícula é dado por *mat* indexando o vetor – **vet[mat] ou vet[9711234]** . Dessa forma, o acesso ao elemento se dá em ordem constante, **imediata...**”

O problema que encontramos é que, nesse caso, o preço pago para se ter esse acesso rápido é muito grande (**consumo de memória**)...”

Tabelas de Dispersão – Funcionamento

“...A ideia central por trás de uma tabela de dispersão é identificar, na chave de busca, quais as partes significativas. Nesse exemplo, o número de matrícula é composto conforme:



Numa turma de alunos, é comum existirem vários alunos com o mesmo ano e período de ingresso. Portanto, esses três primeiros dígitos não são bons candidatos para identificar individualmente cada aluno.

Reduzimos nosso problema para uma chave com os quatro dígitos seqüenciais.

Podemos ir além e constatar que os números seqüenciais mais significativos são os últimos, pois num universo de uma turma de alunos, o dígito que representa a unidade varia mais do que o dígito que representa o milhar...”

Tabelas de Dispersão – Funcionamento

“...Desta forma, podemos usar um número de matrícula parcial, de acordo com a dimensão que queremos que tenha nossa tabela (ou nosso vetor).

Por exemplo, para dimensionarmos nossa tabela com apenas 100 elementos, podemos usar apenas os últimos dois dígitos seqüenciais do número de matrícula. A tabela pode então ser declarada por: ...”

```
struct aluno {  
    int mat;  
    char nome[81];  
    char email[41];  
    char turma;  
};  
typedef struct aluno Aluno;  
  
Aluno* tab[100].
```

Para acessarmos o nome do aluno cujo número de matrícula é dado por mat, usamos como índice da tabela apenas os dois últimos dígitos.

Isso pode ser conseguido aplicando-se o operador modulo (%):

`vet[mat%100] -> nome.`

Tabelas de Dispersão – Funcionamento

“...Como a matrícula é composta por sete dígitos, o número inteiro que conceitualmente representa uma matrícula varia de 0000000 a 9999999. Portanto, precisamos dimensionar nosso vetor com dez milhões (10.000.000) de elementos. Isso pode ser feito por:

```
#define MAX 10000000  
aluno vet[MAX];
```

Tabelas de Dispersão – Funcionamento

*Dessa forma, o nome do aluno com matrícula **mat** é acessado simplesmente por:*

vet[mat] . nome

Temos um acesso rápido, mas pagamos um preço em uso de memória proibitivo. Como a estrutura de cada aluno, no nosso exemplo, ocupa pelo menos 127 bytes, estamos falando num gasto de 1.270.000.000 bytes, ou seja, acima de 1 Gbyte de memória.

*Como na prática teremos, digamos, em torno de 50 alunos cadastrados na turma, precisaríamos apenas de algo em torno de 6.350 bytes na estrutura, proveniente de $(127*50)$ bytes...”*

Tabelas de Dispersão – Funcionamento

*“... A forma de resolver o problema de gasto excessivo de memória, mas ainda garantindo um acesso rápido, é através do uso de **tabelas de dispersão (hash table)**.*

Nota: Utilizando os dois últimos dígitos para identificar um aluno, dizemos que há uma colisão, pois alunos diferentes são mapeados para o mesmo índice da tabela. Para que a estrutura funcione de maneira adequada, temos que resolver esse problema, tratando as colisões. Devemos minimizar as colisões e usar um método que, mesmo com colisões, saibamos identificar cada elemento da tabela individualmente.

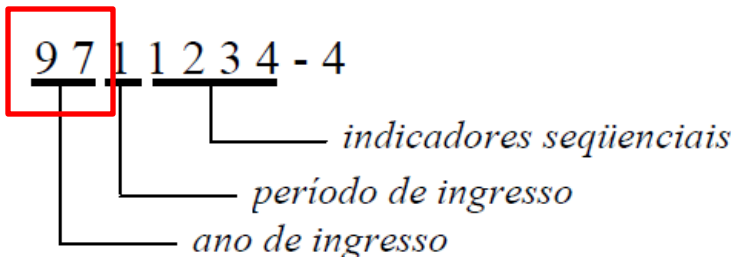
Ainda para minimizarmos o número de colisões, a dimensão da tabela deve guardar uma folga em relação ao número de elementos efetivamente armazenados. Como regra empírica, não devemos permitir que a tabela tenha uma taxa de ocupação superior a 75%. Uma taxa de 50% em geral traz bons resultados. Uma taxa menor que 25% pode representar um gasto excessivo de memória...”

Tabelas de Dispersão – Funcionamento

Uma função de **hash** deve, sempre que possível, apresentar as seguintes propriedades:

- **Ser eficientemente avaliada:** isto é necessário para termos acesso rápido, pois temos que avaliar a função de hash para determinarmos a posição onde o elemento se encontra armazenado na tabela.
- **Espalhar bem as chaves de busca:** isto é necessário para minimizarmos as ocorrências de colisões. O tratamento de colisões requer um procedimento adicional para encontrarmos o elemento. Se a função de hash resulta em muitas colisões, perdemos o acesso rápido aos elementos.

Um exemplo de função de hash ruim seria usar, como índice da tabela, os dois dígitos iniciais do número de matrícula – todos os alunos iriam ser mapeados para apenas três ou quatro índices da tabela...”



```
int hash (int mat)
{
    return (mat%N) ;
}
```

Tabelas de Dispersão – Colisão

```
int hash (int mat)
{
    return (mat%N) ;
}
```

Considerando $N = 100$ e $x=99$, qual será o Índice?

Considerando $N = 100$ e $x=199$, qual será o Índice?

Considerando $N = 100$ e $x=100$, qual será o Índice?

Considerando $N = 100$ e $x=101$, qual será o Índice?

Considerando $N = 100$ e $x=102$, qual será o Índice?

Tabelas de Dispersão – Colisão

“...Existem diversas estratégias para tratarmos as eventuais colisões que surgem quando duas ou mais chaves de busca são mapeadas para um mesmo índice da tabela de hash.

Em todas as estratégias, a tabela de dispersão em si é representada por um vetor de ponteiros para a estrutura que representa a informação a ser armazenada, no caso Aluno. Podemos definir um tipo que representa a tabela por...”

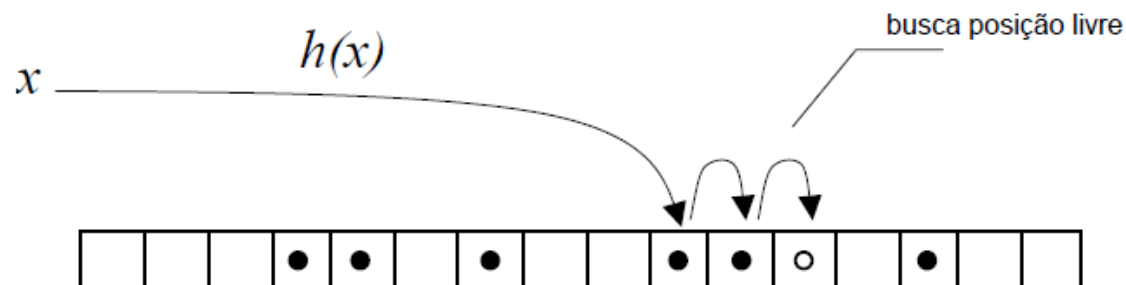
```
#define N 100  
typedef Aluno* Hash[N] ;
```

Tabelas de Dispersão – Colisão

“...Solução 01: Uso da posição consecutiva livre:

*Basicamente nas duas estratégias, **os elementos que colidem são armazenados em outros índices**, ainda não ocupados, da própria tabela.*

Se a função de dispersão mapeia para um índice já ocupado, procuramos o próximo (usando incremento circular) índice livre da tabela para armazenar o novo elemento. Nessa estratégia, os índices da tabela que não têm elementos associados são preenchidos com o valor NULL...”



Tabelas de Dispersão – Colisão

“...Solução 01: Como Inserir ou Modificar:

Fazemos o mapeamento da chave de busca (no caso, número de matrícula) através da função de dispersão e verificamos se o elemento já existe na tabela.

Se o elemento existir, modificamos o seu conteúdo; se não existir, inserimos um novo na primeira posição que encontrarmos na tabela, a partir do índice mapeado...”

```
Aluno* insere (Hash tab, int mat, char* nome, char* email, char
turma)
{
    int h = hash(mat);
    while (tab[h] != NULL) {
        if (tab[h]->mat == mat)
            break;
        h = (h+1) % N;
    }
    if (tab[h]==NULL) { /* não encontrou o elemento */
        tab[h] = (Aluno*) malloc(sizeof(Aluno));
        tab[h]->mat = mat; }
    strcpy(tab[h]->nome,nome);
    strcpy(tab[h]->email,email);
    tab[h]->turma = turma;
    return tab[h];}
```

Tabelas de Dispersão – Colisão

“...Solução 01: Como Localizar:

Devemos notar que a existência de algum elemento mapeado para o mesmo índice não garante que o elemento que buscamos esteja presente. A partir do índice mapeado, temos que buscar o elemento utilizando, como chave de comparação, a real chave de busca, isto é, o número de matrícula completo....”

```
Aluno* busca (Hash tab, int mat)
{
    int h = hash(mat);
    while (tab[h] != NULL) {
        if (tab[h]->mat == mat)
            return tab[h];
        h = (h+1) % N;
    }
    return NULL;
}
```

Tabelas de Dispersão – Colisão

“...Solução 02: dispersão dupla:

*Embora que a solução anterior seja simples, **existe o inconveniente de concentrar os lugares ocupados na tabela, enquanto que o ideal seria dispersar.** Uma estratégia que visa melhorar essa concentração é conhecida como “dispersão dupla” (**double hash**)*

Para evitar a concentração de posições ocupadas na tabela, essa segunda estratégia faz uma variação na forma de procurar uma posição livre a fim armazenar o elemento que colidiu.

Aqui, usamos uma segunda função de dispersão, hash2. Para chaves de busca dadas por números inteiros, uma possível segunda função de dispersão é definida por:

$$\text{hash2}(x) = N - 2 - x \% (N - 2)$$

Tabelas de Dispersão – Colisão

$$\text{hash2}(x) = N - 2 - x \% (N - 2)$$

Considerando $N = 100$ e $x=99$, qual será o Índice?

Considerando $N = 100$ e $x=199$, qual será o Índice?

Considerando $N = 100$ e $x=100$, qual será o Índice?

Considerando $N = 100$ e $x=101$, qual será o Índice?

Considerando $N = 100$ e $x=102$, qual será o Índice?

Tabelas de Dispersão – Colisão

“...Solução 02: Como Localizar:

A implementação da função de busca com essa estratégia é uma pequena variação da função de busca apresentada para a estratégia anterior ...”

```
int hash2 (int mat)
{
    return N - 2 - mat%(N-2);
}

Aluno* busca (Hash tab, int mat)
{
    int h = hash(mat);
    int h2 = hash2(mat);
    while (tab[h] != NULL) {
        if (tab[h]->mat == mat)
            return tab[h];
        h = (h+h2) % N;
    }
    return NULL;
}
```

USO DE LISTAS ENCADEADAS PARA TRATAMENTO DE COLISÕES

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" << endl;
    return 0;
}
```

Uso de Listas Encadeadas

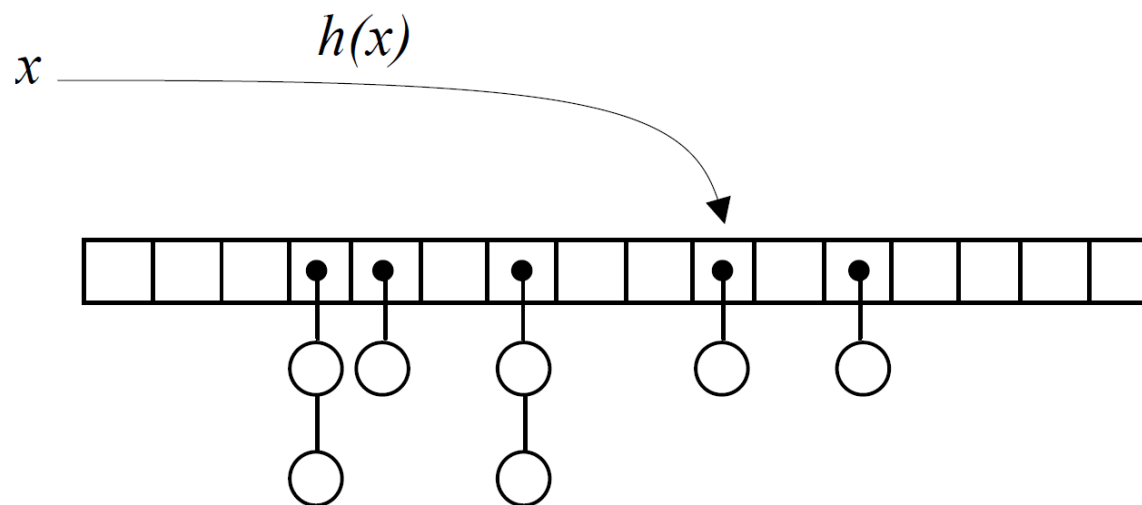
Essa estratégia consiste em fazer com que cada elemento da tabela *hash* represente um ponteiro para uma lista encadeada. Todos os elementos mapeados para um mesmo índice seriam armazenados na lista encadeada.

Os índices da tabela que não têm elementos associados representam listas vazias.

Com essa estratégia, cada elemento armazenado na tabela será um elemento de uma lista encadeada. **Portanto, devemos prever, na estrutura da informação,** um ponteiro adicional para o próximo elemento da lista.

Uso de Listas Encadeadas

Na operação de busca, procuramos a ocorrência do elemento na lista representada no índice mapeado pela função de dispersão.



Implementação

Implemente a função para inserir (ou modificar) um elemento usando a estratégia de uma segunda função de dispersão.

Após modificar o função de inserção, faça um teste de inserção e localização de registros.